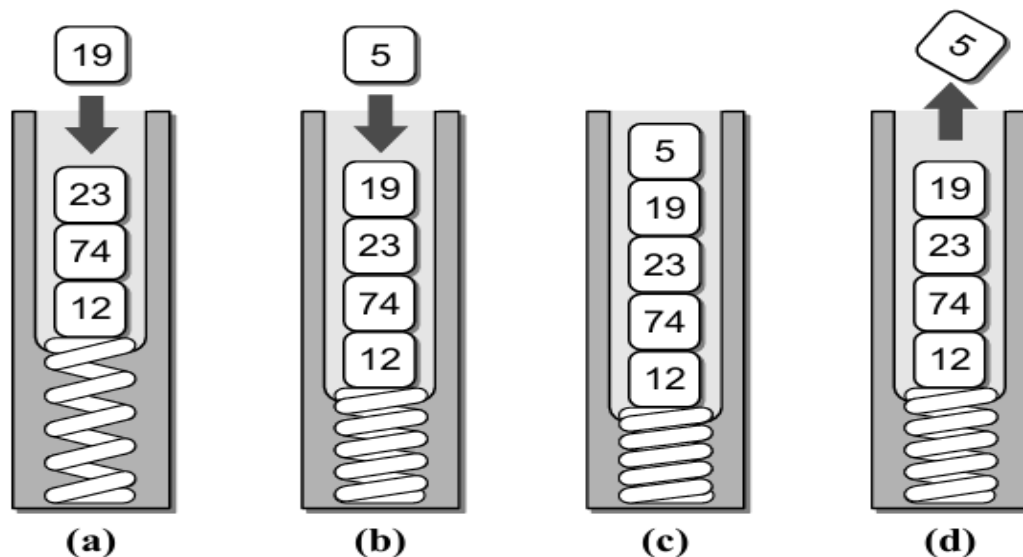


Stack

A **stack** is used to store data such that the last item inserted is the first item removed. It is used to implement a **last-in first-out (LIFO)** type protocol. The stack is a linear data structure in which new items are added, or existing items are removed from the same end, commonly referred to as the **top** of the stack. The opposite end is known as the **base**.



A **stack** is considered a linear data structure because its elements are organized in a **single, sequential order**, typically visualized as a **vertical pile**. This linear arrangement means you can only access elements in a specific order, starting from the top, without **direct access** to elements in the middle or bottom.

1- Operations on Stack

A **stack**, being a linear data structure following the **Last-In-First-Out (LIFO)** principle, supports a **limited** set of operations. Common types of Stack operation includes:

A-Push

Adds an element to the top of the stack.

Example: If the stack is [1, 2, 3] and you push 4, it becomes [1, 2, 3, 4].

B-Pop

Removes and returns the top element from the stack.

Example: If the stack is [1, 2, 3, 4], popping removes 4, leaving [1, 2, 3].

C-Peak or Top

Returns the top element without removing it.

Example: For stack [1, 2, 3, 4], peek returns 4.

D-IsEmpty

Checks if the stack is *empty*.

Returns *true* if no elements exist, *false* otherwise.

E-IsFull

Checks if the stack has reached its *maximum* capacity.

Relevant for array-based stacks with a **predefined size** or in *fixed-size* stack implementations.

F- Size

Returns the *number of elements* currently in the stack.

Example: For stack [1, 2, 3], size returns 3.

These operations ensure the stack maintains its **LIFO behavior**, restricting access to only the top element for simplicity and efficiency. Some implementations may include additional utility operations like **clear** (to remove all elements), **search** (to find an element's position), or **sort** (to arrange elements in ascending or descending order), but the above are the **core operations**.

2- Applications of the Stack

Stacks have various applications across multiple domains, and a few of them are discussed below:

A-Keep tracking of Function Calls (Recursion)

When a program runs functions, a stack keeps track of where to return after each function finishes. It's like leaving breadcrumbs to find your way back.

Example 1:

- 1) Imagine a program where Function A calls Function B, and B calls Function C.
- 2) The stack piles up: first A, then B, then C.
- 3) When C finishes, it's popped off, and the program goes back to B, then A.

Example 2: A game where you pause (Function A), open a menu (Function B), and select an option (Function C). The stack ensures you return to the menu, then the game.

B- Calculating Math Expression

Stacks help solve math problems written in a special format (like postfix: $2\ 3\ +$ for $2 + 3$).

Example:

For $2\ 3\ +\ 4\ *$ (means $(2 + 3) * 4$):

- 1) Push 2, then 3 onto the stack.
- 2) See +: Pop 3 and 2, add them (5), push 5 back.
- 3) Push 4, see *: Pop 5 and 4, multiply (20), push 20.
- 4) Result: 20.

C-Undo/Redo Operations

Stacks allow apps like text editors or drawing programs undo your last action or redo it if you change your mind.

Example:

- 1) You type “Hi” in a text editor, then add “!”.
- 2) Undo stack: [Type Hi, Add !].
- 3) Press Undo: Pop “Add !”, text becomes “Hi”, and “Add !” goes to a redo stack.
- 4) Press Redo: Move “Add !” back to the undo stack, text becomes “Hi!”.

D-Backtracking (Solving Mazes or Puzzles)

Stacks help in problems where you try a path, hit a dead end, and need to go back to try another option, like in mazes or puzzles.

Example:

- 1) In a maze, you move forward and record each step (e.g., “Turn Right”, “Go Straight”).
- 2) Hit a dead end? Pop the last step and try a different direction.
- 3) Stack might look like: [Start, Right, Straight], then pop “Straight” to backtrack.

E- Browser Back/Forward Buttons

Web browsers use stacks to track the pages you visit, so you can go back to the previous page or forward after going back.

Example:

- 1) You visit: Google → YouTube → Wikipedia.
- 2) Back stack: [Google, YouTube, Wikipedia].
- 3) Click Back: Pop Wikipedia, move it to a forward stack → Back stack: [Google, YouTube], Forward stack: [Wikipedia].
- 4) Click Forward: Move Wikipedia back → Back stack: [Google, YouTube, Wikipedia].

Stacks, using the Last-In-First-Out (LIFO) principle, are key in computer science for managing function calls, evaluating expressions, enabling undo/redo, backtracking in mazes, browser navigation, bracket checking, syntax parsing, calculator operations, memory management, graph algorithms like Depth-First Search, and text auto completion, efficiently handling tasks needing reversal or nested tracking.

3- Implementation of Stack

1) First Step is to Make Class of Stack.

```
class Stack:
```

2) Initialization of Constructor

```
def __init__(self, total_values):  
    self.stack = []  
    self.total_values = total_values
```

3) Push Function

```
def push(self):  
    if len(self.stack) == self.total_values:  
        print("Stack is already full, and no space to insert values in it.")  
    else:  
        for i in range(self.total_values - len(self.stack)):  
            element = int(input("Enter the value you want to insert in the Stack: "))  
            self.stack.append(element)  
        print("Stack is: ", self.stack)
```

4) Pop Function

```
def pop(self):  
    if not self.stack:  
        print("Stack is already empty, and there is no value to be removed.")  
    else:  
        e = self.stack.pop()  
        print("Removed value is: ", e)  
        print("Updated Stack is: ", self.stack)
```

5) Peak Function

```
def peak(self):  
    if not self.stack:  
        print("Stack is already empty.")  
    else:  
        print("Top or Peak value of the Stack is: ", self.stack[-1])
```

6) IsFull Function

```
def is_full(self):  
    if len(self.stack) == self.total_values:  
        print("Stack is full.")  
    else:  
        print(f"Stack is not full, and its length is: {len(self.stack)}")
```

7) IsEmpty Function

```
def is_empty(self):  
    if not self.stack:  
        print("Stack is empty.")  
    else:  
        print("Stack is not empty.")
```

8) Size Function

```
def size(self):  
    print(f"Current size of the Stack is: {len(self.stack)}")
```

9) Sort Function

```
def sort(self):  
    if len(self.stack) > 0:  
        self.stack.sort()  
        print("Stack sorted in ascending order and updated sorted stack is: ", self.stack)  
        self.stack.sort(reverse=True)  
        print("Stack sorted in descending order and updated sorted stack is: ", self.stack)
```

10) Display Function

```
def display(self):  
    if not self.stack:  
        print("Stack is already Empty, and no values exist to show.")  
    else:  
        for i in self.stack:  
            print(i)
```

11) Calling Class and Main Menu

```
total_values = int(input("Enter the number of values you want to insert in Stack: "))  
stack = Stack(total_values)  
  
while True:  
    print("\nPlease Enter the Choice:")  
    print("1- Push")  
    print("2- Pop")  
    print("3- Peak")  
    print("4- isFull")  
    print("5- isEmpty")  
    print("6- Size")  
    print("7- Sorting")  
    print("8- Display Stack")  
    print("9- Quit")
```

12) Calling Functions of Stack Class

```
choice = int(input("Please Enter your Choice: "))

if choice == 1:
    stack.push()
elif choice == 2:
    stack.pop()
elif choice == 3:
    stack.peak()
elif choice == 4:
    stack.is_full()
elif choice == 5:
    stack.is_empty()
elif choice == 6:
    stack.size()
elif choice == 7:
    stack.sort()
elif choice == 8:
    stack.display()
elif choice == 9:
    break
else:
    print("Please Enter the correct option to proceed further.")
```

4- Practice Questions

Below are the **5 practice questions** based on the Stack implementation that will help you to learn and practice **Stack** effectively:

1. Browser Navigation Simulation

Write a Python program using a **stack** to simulate **browser navigation**:

- The user can **visit a new page** (push operation).
- The user can **go back** (pop operation).
- Show the **current page** (peak operation).
- Display the full browsing history.

2. Undo Feature in Text Editor

Simulate an **Undo feature** using a stack:

- Each text change should be **pushed** onto the stack.
- When the user presses **Undo**, **pop the last change** and show the previous state.
- Add an option to **view all changes** (display stack).

3. Warehouse Box Management

A warehouse stores boxes in a **stacked manner**:

- Each time a new box arrives, it is **pushed** onto the stack.
- If a box needs to be shipped, **pop** the top box.
- Implement **is_full**, **is_empty**, and **size** to manage warehouse capacity.

4. Balanced Parentheses Checker

Write a program to check if a string of parentheses is **balanced** using a stack:

- Example:
 - Input: $((a+b) * (c+d)) \rightarrow$ Output: Balanced
 - Input: $((a+b) * (c+d) \rightarrow$ Output: Not Balanced

5. Reverse a String using Stack

Write a program that:

- Takes a string input from the user.
- **Pushes each character** onto a stack.
- Pops characters from the stack to **form the reversed string**.
- Displays both **original** and **reversed** strings.